

HOMework 1

LANGUAGE MODELING¹

CS 678 ADVANCED NATURAL LANGUAGE PROCESSING (FALL 2024)

<https://nlp.cs.gmu.edu/course/cs678-fall24/>

OUT: Monday, September 2, 2024

DUE: Friday, September 20, 2024

Your name: Aditya Ketanbhai Shah

Your GID: G01449892

IMPORTANT: This homework is meant to be completed individually.

IMPORTANT: The homework is accompanied by a python notebook. The notebook is here: https://colab.research.google.com/drive/1WhdjWDsqp6nqjVJVCNB1lDftKL0jwd_P?usp=drive_link. After copying this notebook to your Google Drive or One Drive, please paste a link to it below. To get a publicly-accessible link, hit the Share button at the top right, then click "Get shareable link" and copy over the result. Alternatively, you can upload the completed .ipynb file along with the necessary files and your completed .pdf report to Gradescope.

If you fail to do this, you will receive no credit for this homework!

Your Notebook solution: Attached proj1.ipynb file

Graded Questions: 20 points

Total Points Available: 20/20 points

Additional Notes:

- Upload the whole report PDF (including all pages, also if you're scanning it).
- Make sure to share your reproducibility Github repository with the TAs. **If you fail to do this, you will receive no credit for this homework!**
- Some questions might require writing Python code and computing results, and the rest of them have written answers. For coding problems, you will have to fill out all code blocks that say YOUR CODE HERE in the python notebook.
- For text-based answers, you should replace the text that says "Write your answer here..." with your actual answer.

¹Compiled on Wednesday 25th September, 2024 at 22:16

Follow the instructions from the notebook. To streamline our grading, please provide selected outputs from your notebook below, and answer the questions listed here.

1 n -gram Language Models (6 points)

1. (1 point) Copy the sampled output produced by your unigram model.

```
arrows @-@ and styles but best method Hanuman was ground a of to mushrooms Neminatha also Aquitania their  
SO Azuma
```

2. (2 points) What is the validation perplexity of your bigram model?

```
perplexity =  
635.6280341733934
```

Don't forget to also submit your `bigram_predictions.npy` file.

3. (3 points) What is the validation perplexity of your trigram backoff model?

```
perplexity =  
310.72627114846125
```

Don't forget to also submit your `trigram_backoff_predictions.npy` file.

2 Neural Language Models (6+8 points)

4. (2 points) What is the validation perplexity of your neural trigram model?

```
perplexity =  
240.37557788213746
```

Don't forget to also submit your `neural_trigram_predictions.npy` file.

5. (4 points) What is the validation perplexity of your neural LSTM model?

```
perplexity =  
156.40791950612498
```

2.1 Experimentation (8+2 points)

Now it's time for you to experiment. Try to reach a validation perplexity below 200 or even 160. (BONUS: Lowest two perplexities in class get bonus +2 points)

It is okay if the bulk of your improvements are due to hyperparameter tuning (such as changing number or sizes of layers), but implement at least one more substantial change to the model. Here are some

ideas (several of which come from <https://arxiv.org/pdf/1708.02182.pdf>):

- activation regularization - add a l2 regularization penalty on the activation of the LSTM output (standard l2 regularization is on the weights)
- weight-drop regularization - apply dropout to the weight matrices instead of activations
- learning rate scheduling - decrease the learning rate during training
- embedding dropout - zero out the entire embedding for a random set of words in the embedding matrix
- ensembling - average the predictions of several models trained with different initialization random seeds
- temporal activation regularization - add l2 regularization on the difference between the LSTM output activations at adjacent timesteps

You may notice that most of these suggestions are regularization techniques. This dataset is considered fairly small, so regularization is one of the best ways to improve performance.

6. (2 points) What is the validation perplexity of your improved LSTM model?

perplexity	=
115.44008324646242	

Don't forget to also submit your `lstm_predictions.npy` file.

7. (6 points) Describe the extensions and/or modifications that you tried. Your description should be **1.5-page maximum** in length (including figures or tables). For full credit, your write-up should include:

1. A concise and precise description of the extension that you tried.
2. A motivation for why you believed this approach might improve your model.
3. A discussion of whether the extension was effective and/or an analysis of the results. This will generally involve some combination of tables, learning curves, etc.
4. A bottom-line summary of your results comparing validation perplexities of your improvement to the original LSTM.

Write your answer below. Make sure to not exceed page 5 of this document. (Do remove the "pagebreaks" below and the note.

Extensions and Modifications Implemented:

I applied the following changes to the LSTM model:

- **Gradient Clipping:** Added to control the maximum gradient values during backpropagation.
- **Weight Decay:** Used in the optimizer to prevent large weights and overfitting.
- **Increased Sequence Length:** Raised the sequence length from 32 to 64 for capturing longer dependencies.
- **Activation Regularization:** Continued using L2 regularization on activations.
- **Learning Rate Scheduling:** Reduced the learning rate automatically when validation perplexity plateaued.

Motivation:

- **Gradient Clipping:** To prevent exploding gradients and make training more stable.
- **Weight Decay:** To reduce overfitting and improve validation performance.
- **Increased Sequence Length:** To give the model more context, improving predictions.
- **Learning Rate Scheduling:** To help the model converge better by lowering the learning rate when needed.

Effectiveness and Results:

- **Training:** Training loss decreased consistently across epochs.
- **Validation:** Validation perplexity improved significantly.
- **Gradient Clipping:** Stabilized training by preventing large spikes in loss.
- **Weight Decay:** Reduced overfitting, resulting in lower validation perplexity.
- **Increased Sequence Length:** Gave better predictions by considering more context but added some computational cost.
- **Learning Rate Scheduling:** Helped with better convergence when improvements slowed.

Bottom-line Summary:

The modifications—including gradient clipping, weight decay, Activation regularization, learning rate

scheduling and increasing sequence length—helped lower the validation perplexity from ~166 in the original model to 115.44 after 20 epochs in the improved model. These changes significantly enhanced training stability, reduced overfitting, and allowed the model to capture longer-term dependencies. Over the course of training, validation perplexity steadily decreased, demonstrating the effectiveness of these adjustments.

```
Epoch 1/20
Validation Perplexity after Epoch 1: 571.20
Epoch 2/20
Validation Perplexity after Epoch 2: 398.22
Epoch 3/20
Validation Perplexity after Epoch 3: 308.71
Epoch 4/20
Validation Perplexity after Epoch 4: 245.92
Epoch 5/20
Validation Perplexity after Epoch 5: 210.58
Epoch 6/20
Validation Perplexity after Epoch 6: 186.34
Epoch 7/20
Validation Perplexity after Epoch 7: 169.57
Epoch 8/20
Validation Perplexity after Epoch 8: 158.46
Epoch 9/20
Validation Perplexity after Epoch 9: 149.14
Epoch 10/20
Validation Perplexity after Epoch 10: 141.80
Epoch 11/20
Validation Perplexity after Epoch 11: 137.06
Epoch 12/20
Validation Perplexity after Epoch 12: 132.37
Epoch 13/20
Validation Perplexity after Epoch 13: 128.43
Epoch 14/20
Validation Perplexity after Epoch 14: 125.49
Epoch 15/20
Validation Perplexity after Epoch 15: 124.13
Epoch 16/20
Validation Perplexity after Epoch 16: 120.77
Epoch 17/20
Validation Perplexity after Epoch 17: 118.81
Epoch 18/20
Validation Perplexity after Epoch 18: 118.04
Epoch 19/20
Validation Perplexity after Epoch 19: 117.29
Epoch 20/20
Validation Perplexity after Epoch 20: 115.44

LSTM validation perplexity: 115.44008324646242
```

List of Deliverables

A .zip file, named as ``your_email_ID`.zip` (e.g., the instructor would submit a `antonis.zip` file), including the following:

- `bigram_predictions.npy`
- `trigram_backoff_predictions.npy`
- `neural_trigram_predictions.npy`
- `lstm_predictions.npy` (this should involve your improvements)
- `proj_1.ipynb` (the completed notebook. Rename it to this *exactly*)
- `report.pdf`

Collaboration Questions

Please answer the following:

1. Did you receive any help whatsoever from anyone in solving this assignment?
Yes / No.
 - If you answered 'yes', give full details: No
 - (e.g. "Jane Doe explained to me what is asked in Question 3.4")
2. Did you give any help whatsoever to anyone in solving this assignment?
Yes / No.
 - If you answered 'yes', give full details: _____
 - (e.g. "I pointed Joe Smith to section 2.3 since he didn't know how to proceed with Question 2")
3. Did you find or come across code that implements any part of this assignment ?
Yes / No. (See below policy on "found code")
 - If you answered 'yes', give full details: _____
 - (book & page, URL & location within the page, etc.).

References:

1. Neural N-Gram Architecture and Implementation:

- Dahami, K. (n.d.). *A Neural Probabilistic Language Model: Breaking Down Bengio's Approach*. Medium.
- URL: [Neural N-Gram Model](#)

2. Neural N-Gram Model: Implementation and Training:

- daandouwe. (n.d.). *Neural N-Gram Model - GitHub Repository*. GitHub.
- URL: [Neural N-Gram Model - GitHub](#)

3. LSTM Architecture and Implementation:

- Tang, Y. (n.d.). *LSTM Language Model Example in PyTorch*. GitHub.
- URL: [LSTM Implementation](#)

4. Gradient Clipping:

- Pascanu, R., Mikolov, T., & Bengio, Y. (2013). *On the Difficulty of Training Recurrent Neural Networks*. In International Conference on Machine Learning (pp. 1310-1318).
- **URL:** [Proceedings of ICML 2013](#)
- **Location:** Section on *gradient clipping*.

5. Perplexity Evaluation:

- Jurafsky, D., & Martin, J. H. (2009). *Speech and Language Processing* (2nd edition). Prentice Hall.
- **Location:** Chapter discussing *language model evaluation*.